WEEK 4 · LESSON 23 OF 24 · HANDOUT

Capstone Build: An End-to-End Reference Project

Study time: **about 150 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=KGpg8jwAda4 — open the lesson online to watch inside the course.

🎓 **Go deeper:** EuroPython 2020: Painless Machine Learning in Production — https://www.youtube.com/watch?v=8Zn4hV9xE1I

🎓 **Go deeper:** PyCascades 2024: Embracing MLOps — Bridging the Gap Between Machine Learning and DevOps — https://www.youtube.com/watch?v=wRnKcWIZEvk

Your goal in this lesson: Follow a complete reference implementation — validated data, tracked training, quality and fairness gates, documentation, a secure API with monitoring, and tests — and adapt it to your own capstone.

The reference system

This notebook builds the whole SACCO system in one run. Use it as a **template**: replace the data and features with your own, keep the structure.

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

Step 1 — Settings and data checks

```
import hashlib, json, os, pathlib, secrets, time
import joblib
from collections import defaultdict, deque
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

CONFIG = {"version": "2.0.0", "features": ["income_rwf", "loan_rwf", "years_member",
"late_payments", "salaried"],
          "target": "repaid", "seed": 0, "threshold": 0.5, "review_band": [0.35, 0.65],
          "gates": {"min_auc": 0.80, "min_disparate_impact": 0.80}}
F, T = CONFIG["features"], CONFIG["target"]
OUT = pathlib.Path("capstone"); OUT.mkdir(exist_ok=True)

assert applicants[F + [T]].notna().all().all(), "missing values"
assert applicants["income_rwf"].gt(0).all() and applicants["loan_rwf"].gt(0).all(), "bad
amounts"
print("data checks passed")
```

Step 2 — Train and track

```
tr, te = train_test_split(applicants, test_size=0.25, random_state=CONFIG["seed"],
stratify=applicants[T])
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(tr[F], tr[T])
scores = model.predict_proba(te[F])[:, 1]
approved = (scores >= CONFIG["threshold"]).astype(int)
metrics = {"auc": round(roc_auc_score(te[T], scores), 3), "accuracy":
round(accuracy_score(te[T], approved), 3)}
fingerprint = hashlib.sha256(pd.util.hash_pandas_object(tr,
index=False).values.tobytes()).hexdigest()[:16]
with open(OUT / "runs.jsonl", "a") as f:
    f.write(json.dumps({"config": CONFIG, "metrics": metrics, "data": fingerprint}) + "\n")
print(metrics)
```

Notice version 2.0.0 drops mobile-money activity, following the Week 3 audit.

Step 3 — Quality and fairness gates

```
def disparate_impact(data, group, decision):
    rates = data.assign(d=decision).groupby(group)["d"].mean()
    return round(float(rates.min() / rates.max()), 3)

fair = {g: disparate_impact(te, g, approved) for g in ["area", "gender"]}
gates = {f"AUC {metrics['auc']} >= {CONFIG['gates']['min_auc']}": metrics["auc"] >=
CONFIG["gates"]["min_auc"]}
gates.update({f"disparate impact by {g} {v} >= {CONFIG['gates']['min_disparate_impact']}": v >=
CONFIG["gates"]["min_disparate_impact"]
              for g, v in fair.items()})
for name, ok in gates.items():
    print("PASS" if ok else "FAIL", name)
RELEASE = all(gates.values())
print("release allowed" if RELEASE else "release BLOCKED")
```

Step 4 — Save the artifact and the model card

```
joblib.dump(model, OUT / "model.joblib")
meta = {**CONFIG, "metrics": metrics, "fairness": fair, "data_fingerprint": fingerprint,
"released": RELEASE}
(OUT / "metadata.json").write_text(json.dumps(meta, indent=2))
(OUT / "MODEL_CARD.md").write_text("\n".join([
    f"# Model card: loan-repayment v{CONFIG['version']}",
    "Intended use: support loan officers; scores in the review band go to a person; no
automatic refusals.",
    f"Features: {', '.join(F)} (no gender, area, age or mobile-money activity).",
    f"Performance: AUC {metrics['auc']}, accuracy {metrics['accuracy']}.",
    f"Disparate impact: {fair}.",
    "Limits: monitor concept drift monthly; human review of all refusals.",
]))
print(sorted(p.name for p in OUT.iterdir()))
```

Step 5 — A secure, monitored API

```
from fastapi import Depends, FastAPI, Header, HTTPException
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field

KEY = os.environ.get("CAPSTONE_API_KEY") or secrets.token_hex(16)
SALT = os.environ.get("LOG_SALT", "demo-salt").encode()
LOGS, calls = [], defaultdict(deque)
scorer = joblib.load(OUT / "model.joblib")

class Applicant(BaseModel):
    applicant_id: int
    income_rwf: float = Field(..., ge=10_000, le=20_000_000)
    loan_rwf: float = Field(..., ge=10_000, le=5_000_000)
    years_member: int = Field(..., ge=0, le=60)
    late_payments: int = Field(..., ge=0, le=50)
    salaried: int = Field(..., ge=0, le=1)

def auth(x_api_key: str = Header(default="")):
    if not secrets.compare_digest(x_api_key, KEY):
        raise HTTPException(401, "Missing or wrong API key")
    q, now = calls[x_api_key], time.time()
    while q and now - q[0] > 60:
        q.popleft()
    if len(q) >= 30:
        raise HTTPException(429, "Too many requests")
    q.append(now)

app = FastAPI(title="SACCO loan support", version=CONFIG["version"])

@app.get("/health")
def health():
    return {"status": "ok", "version": CONFIG["version"]}

@app.post("/decide", dependencies=[Depends(auth)])
def decide(a: Applicant):
    start = time.perf_counter()
    p = float(scorer.predict_proba(pd.DataFrame([a.model_dump()])[F])[0, 1])
    low, high = CONFIG["review_band"]
    route = "approve" if p >= high else "human review"           # no automatic refusals
    LOGS.append({"who": hashlib.sha256(SALT + str(a.applicant_id).encode()).hexdigest()[:12],
                 "score": round(p, 3), "route": route, "ms": round((time.perf_counter() -
start) * 1000, 2),
                 "version": CONFIG["version"]})
    return {"route": route, "version": CONFIG["version"]}

@app.get("/metrics", dependencies=[Depends(auth)])
def metrics_endpoint():
    df = pd.DataFrame(LOGS)
    if df.empty:
        return {"requests": 0}
    return {"requests": len(df), "approve_rate": round(float((df["route"] ==
"approve").mean()), 3),
            "p95_ms": round(float(df["ms"].quantile(0.95)), 2)}
```

Step 6 — Test everything

```
api = TestClient(app)
H = {"X-API-Key": KEY}
sample = te.head(20)
for _, r in sample.iterrows():
    body = {"applicant_id": int(r["applicant_id"]), **{f: (int(r[f]) if f in ("years_member",
"late_payments", "salaried") else float(r[f])) for f in F}}
    api.post("/decide", json=body, headers=H)

tests = {
    "health works": api.get("/health").status_code == 200,
    "no key is refused": api.post("/decide", json=body).status_code == 401,
    "bad input is refused": api.post("/decide", json={**body, "income_rwf": 5},
headers=H).status_code == 422,
    "no automatic refusals": all(l["route"] in ("approve", "human review") for l in LOGS),
    "logs hold no raw IDs": all(str(int(r["applicant_id"])) != l["who"] for l, (_, r) in
zip(LOGS, sample.iterrows())),
    "metrics endpoint works": api.get("/metrics", headers=H).json()["requests"] == 20,
    "gates passed": RELEASE,
}
for name, ok in tests.items():
    print(("PASS  " if ok else "FAIL  ") + name)
print(api.get("/metrics", headers=H).json())
```

Adapting it to your capstone

Keep	Replace
The seven-step structure	Data, features, target
Gates (quality + fairness)	Thresholds that fit your problem
Model card, metadata, run log	The text describing your use and limits
API key, rate limit, validation, pseudonymised logs	Field names and limits
Tests	Add tests for your own behaviour rules

Then add the Docker package from Lesson 5 and the CI workflow from Lesson 7.

Remember this: A capstone-quality AI system is a chain — checked data, tracked training, gates, documentation, a secure monitored API and tests — and this template gives you every link.

Name: _____ Date: _____

Activity

Task 1

Run the template with the mobile-money feature added back. Which gate changes?

Task 2

Change the review band to 0.45–0.55. What happens to the approve rate in `/metrics`?

Task 3

Replace the SACCO data with your own capstone data and get all tests to pass.

Task 4

Add a Dockerfile and a GitHub Actions workflow to the `capstone/` folder.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why does version 2.0.0 drop the mobile-money feature?

   A. Following the Week 3 audit findings on fairness and gaming
   B. It was always missing
   C. By accident
   D. To save memory

2. What happens if a gate fails in Step 3?

   A. The data is deleted
   B. The release is blocked
   C. The API restarts
   D. The model is released anyway

3. What does the capstone API return for uncertain or low scores?

   A. Decline
   B. Human review
   C. Nothing
   D. An error

4. Why do the logs store a salted hash instead of the applicant ID?

   A. IDs are wrong
   B. It is faster
   C. Hashes are shorter
   D. To protect privacy while still allowing linking

5. Which status code does the API return without an API key?

   A. 422
   B. 401
   C. 200
   D. 429

6. What should you keep when adapting the template?

   A. The structure: checks, gates, documentation, secure API and tests
   B. Only the model
   C. Only the SACCO data
   D. Nothing

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.